Exercício - Chamados de Suporte Técnico (com gravação em arquivo)

Objetivo

Implementar um mini–service desk que registre um Chamado (ticket) com prioridade, associe-o a um Cliente (composição), encaminhe para um Atendimento (interface), e grave as informações do chamado resolvido em um arquivo chamados.txt.
O programa deve também tratar exceções (try-catch) e utilizar um enum para representar prioridades.

Requisitos obrigatórios

1. **Enum**: representar **níveis de prioridade** do chamado.
2. **Interface**: Atendimento com método resolverChamado(Chamado chamado).
3. **Composição**: Chamado **possui** um Cliente.
4. **Try-catch**: tratar entrada inválida e erros de gravação de arquivo.
5. **Persistência**: registrar o resultado do atendimento no arquivo chamados.txt.
6. **Execução em um único arquivo .java**.

Contexto

Você foi contratado para desenvolver um sistema de Suporte Técnico. Cada cliente abre um chamado com uma prioridade (BAIXA, MEDIA, ALTA, CRITICA).
Chamados BAIXA e MEDIA são resolvidos pelo Suporte Básico, enquanto ALTA e CRITICA são resolvidos pelo Suporte Avançado.

Ao final, o programa deve registrar o chamado resolvido em um arquivo chamados.txt, com informações formatadas como:

ID: 1 | Cliente: Ana | Prioridade: ALTA | Resolução: Troca de fonte de alimentação

Etapas de Desenvolvimento

1 Criar o enum NivelPrioridade

- Constantes: BAIXA, MEDIA, ALTA, CRITICA
- (Opcional) Método getDescricao() retornando um texto mais amigável.

2 Criar a interface Atendimento

- Método:
- String resolverChamado(Chamado chamado);
- Deve retornar uma string descrevendo a ação realizada.

3 Implementar as classes concretas

- SuporteBasico e SuporteAvancado implementam Atendimento.
- Retornar mensagens distintas para cada tipo de suporte.

4 Criar as classes de modelo com composição

- Cliente: atributos nome e email.
- Chamado: atributos id, descricao, prioridade (enum) e cliente.

5 Adicionar método validar() em Chamado

Verifica:

- cliente não nulo;
- nome, email e descricao não vazios;
- prioridade diferente de null.

Se inválido, lançar IllegalArgumentException.

6 Escolher atendimento conforme prioridade

- BAIXA e MEDIA → SuporteBasico
- ALTA e CRITICA → SuporteAvancado

7 Implementar gravação do chamado

Após resolver o chamado, gravar a linha no arquivo chamados.txt:

ID: <id> | Cliente: <nome> | Prioridade: <prioridade> | Resolução: <mensagem>

Use FileWriter ou PrintWriter em modo de **acréscimo (true)**, e **trate exceções com try-catch**.

Exemplo:

```
try (FileWriter fw = new FileWriter("chamados.txt", true);
     PrintWriter pw = new PrintWriter(fw)) {
    pw.println(linha);
} catch (IOException e) {
    System.out.println("Erro ao gravar o chamado: "+ e.getMessage());
}
```

8 Interação com o usuário

O programa deve:

1. Exibir as prioridades disponíveis (NivelPrioridade.values());
2. Ler nome, e-mail, descrição e prioridade do usuário;
3. Criar o Chamado, validar, encaminhar para o atendimento correto, resolver e gravar no arquivo;
4. Exibir mensagem de sucesso ou erro.

9 Exemplo de saída

```
Prioridades: BAIXA MEDIA ALTA CRITICA

Nome do cliente: Nilton

E-mail: nilton@mack.com.br

Descrição do chamado: "instalação de novo software"

Prioridade (BAIXA, MEDIA, ALTA, CRITICA): ALTA


ID: 1 | Cliente: Nilton | Email: nilton@etapa.com.br | Prioridade:
ALTA | Resolução: [Suporte Avançado] Chamado #1 (ALTA) resolvido:
Troca de hardware / intervenção remota avançada..
```